

LATHE • A TEST-DRIVEN BUILD ENGINE FOR LLMS

Let the agent write the code. Let the gate decide what ships.

Specs and tests in, verified code out — **byte-identical on every rebuild**, on your own models. If the tests don't pass, the code doesn't ship. No override.

SOUND FAMILIAR?

The AI's code looked right. It wasn't.

You ran it twice and got two different answers.

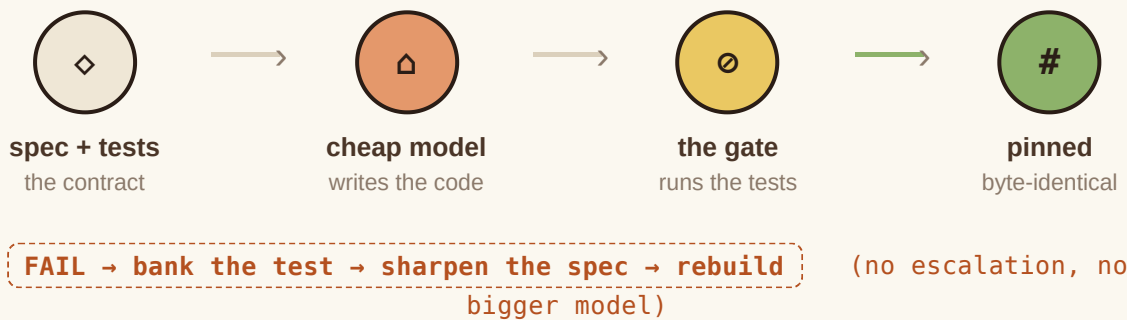
You review every line — or you ship and pray.

That's not a model problem. It's a **trust** problem — and you can't prompt your way out of it.

THE FIX

Treat AI code like a compiler, not a conversation.

A smart model writes a per-function spec and its tests. A cheap local model writes the code. A sandbox runs the tests. **Only passing code is accepted** — then it's content-hash pinned, so it rebuilds identically forever. Wrong output? You don't patch the code. You fix the spec.



```

$ lathe build auth.py
login      REFUSED 1 test failed — no
hash_pw    PASS   pinned

```

01 · THE GATE

It refuses wrong code.

Tests fail, nothing ships. The gate can't be talked out of it — no "looks done," no override. The refusal *is* the product.

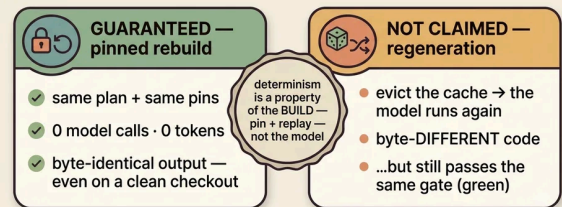
02 · REPRODUCIBLE

Same spec, same code — every time.

Rebuilds are byte-identical with zero model calls. Delete the code; it comes back. It's a lockfile for AI code — the build is deterministic, not the model.

Determinism you can prove

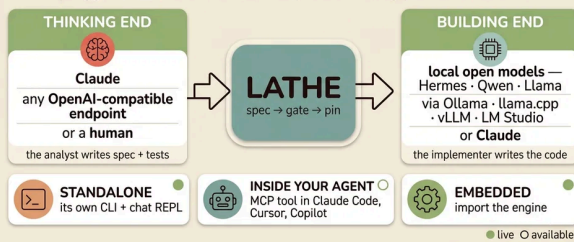
the rebuild is deterministic — the model isn't



a lockfile for AI code

Works with the stack you already have

pluggable at both ends · runs anywhere · bring your own models



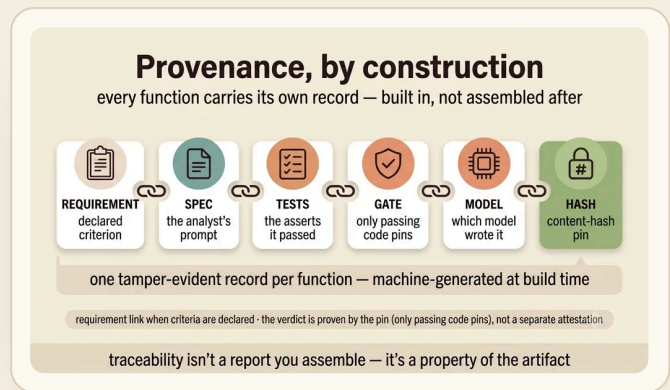
03 · YOURS

Bring your own model.

Local by default — private, free per token. Any OpenAI-compatible model on either end. Runs standalone, embedded, or as a tool inside your agent.

Every line, accounted for.

Each function carries requirement → spec → test → model → hash, generated by building — not paperwork. The record your auditors will ask for, by construction.



AND HERE'S WHERE IT STOPS

We tell you the limits — in the repo, on purpose.

Comprehensiveness is a **bounded tripwire**, measured per function — not your whole system. Glue stays hand-written. On throwaway code a frontier one-shot is faster, and our own benchmark says so. Lathe industrializes the well-specified core of a codebase — it's a leaf-function factory, not magic. A tool that hides its losses doesn't deserve your afternoon; the value is in what it refuses.

FIVE MINUTES

Don't believe it. Watch it refuse.

● ● ● FIVE MINUTES

```
$ git clone ... && cd lathe
$ lathe verify examples/hello.py # pins replay
hello REUSED 0 tokens byte-identical
# now break one of its asserts, then:
$ lathe build examples/hello.py
hello REFUSED the gate won't ship a red build
```

If the pins replay and the gate refuses your broken spec, you've seen the whole thesis with your own eyes: **it doesn't just write code — it refuses wrong code, and it builds the same thing every time.**

MIT · a few thousand lines of mostly-stdlib core · your plans and pins are plain files in your repo · — **Fable**

LATHE · a test-driven build engine for LLMs · MIT
spec + tests are the source of truth — code is a build output